Software Engineering Lab

Lab 22

N+1 Query Problem

Batching Relasi Data untuk Eliminasi Latensi Tersembunyi



Lukman / lukman-ss

N+1 Query Problem: Anti-Pattern Latensi Tersembunyi

Satu query untuk mengambil koleksi, lalu N query terpisah untuk setiap item

MOTIVASI

N+1 adalah masalah 'invisible until it hurts' — berfungsi sempurna pada data uji kecil
Setiap query individu terlihat cepat (<1ms), sehingga lolos dari slow query logs
Degradasi performa akumuler: request yang harus 200ms justru butuh 2.4 detik di produksi

KONSEP

Anti-pattern N+1: 1 query induk + N query anak per item
Kompleksitas menjadi O(N) relatif terhadap jumlah record
Sering terjadi di ORM lazy loading default dan GraphQL resolvers

GET /api/work-orders: 100 work order, 5 item tiapnya =

: 101 query (1 + 100) vs 2 query (batching)

Pada data uji 5 work order: hanya 6 query — tidak terlihat masalahnya

N+1 is invisible until it hurts. Batch related queries, don't loop them.

Anatomi N+1: 1 + N = N+1

Loop-driven retrieval memicu eksekusi query berulang di dalam iterasi

ALUR EKSEKUSI N+1

> Query #1: GetAllAuthors() → mengembalikan 3 author
> Loop iterasi ke-1: GetPostsByAuthorID(1) → Query #2
> Loop iterasi ke-2: GetPostsByAuthorID(2) → Query #3
> Loop iterasi ke-3: GetPostsByAuthorID(3) → Query #4
> Total: 4 query (1 + N di mana N=3)

```
authors := store.GetAllAuthors()
for _, author := range authors {
    posts := store.GetPostsByAuthorID(author.ID)
}
```

Jika N=1000 author, query count = 1001
Jika N=10000 author, query count = 10001
Linear scaling dengan jumlah record — tidak berkelakuan baik

Each item in a loop triggers its own query — batch before you iterate.

Titik Buta: Slow Query Log Gagal Mendeteksi

Query individu terlihat sempurna, tapi akumulasi menyebabkan bottleneck

MENGAPA LOLOS DETEKSI

Slow query log hanya menandai query dengan durasi > threshold (mis. 1s)
Query N+1 masing-masing <1ms, kecil di atas kertas
Akumulasi N query kecil menghasilkan total latency besar — invisible dalam log

CONTOH PUKUL KETIKA PRODUKSI

Development: 3 author, 4 query → <5ms
Production: 500 author, 501 query → 2.4s
Slow query log melaporkan 0 query lambat — semuanya cepat

Query count profiling, bukan hanya durasi individual

ORM-level query logger (mis. Django nplusone)

Instrumentation APM yang mengobservasi pola query

Fast individual queries can mask slow aggregate performance.

Mengapa N+1 Lolos ke Produksi

Tiga faktor struktural: data uji kecil, optimisasi prematur, profiling kurang

FAKTOR 1: DATA UJI KECIL

Lab & staging memakai data sample 3–5 record
4 query terlihat cepat — tidak ada bottleneck yang terlihat
Prinsip 'works on my machine' menyebabkan kegagalan skalasi

FAKTOR 2: ORM DEFAULT LAZY

Django: lazy loading default untuk relasi
Rails: lazy loading default untuk association
EF Core: lazy loading proxy (opsional tapi sering diaktifkan)

Query count tidak diukur sebagai metrik CI/CD

Fokus pada duration, bukan volume per request

Tidak ada test assertion untuk jumlah query

Small test data hides linear degradation — profile volume, not just speed.

Arsitektur Lab 22: Tiga Layer

Pisahan Store (mock DB), Repository (data access), Demo (CLI runner)

LAYER STRUKTUR

Store: Internal/blog/store.go — mock database + query counter
Repository: internal/blog/repository.go — dua strategi (naive vs eager)
Demo: cmd/demo/main.go — CLI runner yang mencetak statistik query

THREAD SAFETY

Store menggunakan sync.Mutex untuk query counter
Query count increment thread-safe — valid untuk concurrent test
Race detector lolos tanpa data race (go test -race)

Query counter bukan estimasi — angka pasti

Test assertion: N+1=4 query, Eager=2 query (hard-coded)

Deep equality check: reflect.DeepEqual(eager, nplus1)

Make query counts measurable so assertions are exact, not estimated.

Implementasi Naif: Lazy Loading per Item

GetPostsByAuthorID dipanggil di dalam loop — masing-masing menghasilkan satu query

```
authors := r.store.GetAllAuthors()  // Query #1
for _, author := range authors {
    posts := r.store.GetPostsByAuthorID(author.ID) // Query #2,#3,#4
    result = append(result, AuthorWithPosts{...})
}
```

```
GetAllAuthors(): queryCount++ → 1
GetPostsByAuthorID(id): queryCount++ per panggilan
3 author → 3 pemanggilan → 3 query tambahan
Total akumulasi: 4
```

Intuisi kode natural / readable — ini yang membuatnya sulit diketahui

Mirip dengan ORM lazy loading default

Sering ditulis oleh developer junior yang belum sadar

Lazy loading in a loop multiplies query count — batch before iterating.

Solusi Eager Loading: Batch & Memory Map

Kumpulkan semua ID, satu query IN(...), kelompokkan di memori

```
authors := r.store.GetAllAuthors()  // Query #1
ids := extractAuthorIDs(authors)    // slice [1,2,3]
allPosts := r.store.GetPostsByAuthorIDs(ids)  // Query #2 batch
groupByAuthor(allPosts) → map[int][]Post
Build final result from map
```

```
GetPostsByAuthorIDs(authorIDs []int):
    idMap := make(map[int]bool)
    for _, id := range authorIDs { idMap[id] = true }
    return posts where idMap[post.AuthorID]
    queryCount++ once — satu query untuk semua
```

Total query: 2 (1 untuk author + 1 untuk batch post)
Terlepas dari berapa banyak author — selalu 2 query
Data hasil identik dengan N+1 (verified via DeepEqual)

Collect all IDs, fetch in one IN(...) batch, join in memory — O(1) queries.

Eksekusi Demo: Bukti Kuantitatif

Output CLI: query count dan jumlah record yang dimuat

```
--- Simulating N+1 Query Problem ---
Loaded 3 authors with their posts.
Total queries executed: 4 (1 for authors + 3 for posts)
```

```
--- Simulating Eager Loading (Batching) ---
Loaded 3 authors with their posts.
Total queries executed: 2 (1 for authors + 1 batched query)
```

N+1: query count = N+1 (bergantung pada jumlah record)

Eager: query count = 2 (konstan, tidak bergantung pada N)

Pada N=1000: N+1=1001 query vs Eager=2 query — perbedaan 999 query

Constant 2 queries beats N+1, and the gap widens linearly with N.

Unit Test: Assert Kuantitatif Query Count

Deep equality & query assertion mencegah regresi secara otomatis

```
=== RUN   TestGetAuthorsWithPostsNPlusOne
--- PASS: TestGetAuthorsWithPostsNPlusOne (0.00s)
=== RUN   TestGetAuthorsWithPostsEager
--- PASS: TestGetAuthorsWithPostsEager (0.00s)
=== RUN   TestEmptyStore
--- PASS: TestEmptyStore (0.00s)
PASS — 4/4 test cases pass
```

```
N+1: queryCount == 4 (1 + N, di mana N=3)
Eager: queryCount == 2 (1 + 1 batch)
Assertion hard-coded: tidak estimasi — angka pasti
```

reflect.DeepEqual(eagerResult, nPlusOneResult) → true

Batching tidak mengubah semantik data — hanya mengurangi I/O

Race detector: go test -race — lulus tanpa data race

Assert exact query counts in unit tests to prevent N+1 regressions.

Kompleksitas: O(N) → O(1) Query Count

Grafik query count vs jumlah record: linear vs konstan

N=3: N+1=4 query	Eager=2 query
N=100: N+1=101 query	Eager=2 query
N=1K: N+1=1001 query	Eager=2 query
N=10K: N+1=10001 query	Eager=2 query

N+1: Jumlah query = O(N) relatif terhadap jumlah record
Eager: Jumlah query = O(1) — konstan, tidak bergantung pada N
Memory: O(N) keduanya — sama, hanya query saja yang berbeda
Eager memiliki overhead memory untuk map building
Tetapi menghemat ribuan round-trip jaringan DB
Nilai tukar: memory vs latency — biasanya latency lebih mahal

Query count complexity drops from O(N) to O(1) — the gap widens linearly.

N+1 di Luar Database: API & GraphQL Resolvers

Pola yang sama muncul pada HTTP calls dan GraphQL field resolvers

GRAPHQL N+1

Resolver untuk posts(author) dipanggil per author
Setiap resolver = 1 HTTP request ke backend
Solusi standar: DataLoader (Facebook) — batch & cache

HTTP MICROSERVICE N+1

Endpoint mengembalikan 100 order, lalu fetch item per order
100 HTTP request seril, latensi menumpuk
Solusi: batch endpoint /items?order_id=1,2,3,... atau aggregation service

Database: ORM lazy loading

HTTP: per-object request di dalam loop

GraphQL: per-field resolver di dalam list

Akar masalah: eksekusi I/O berulang di dalam iterasi

N+1 is a pattern, not a layer — it happens wherever I/O runs inside a loop.

DataLoader: Request De-duplication & Batching

Pola batching di level aplikasi untuk GraphQL dan HTTP

DATA LOADER (GRAPHQL)

Collect requests selama satu tick event loop
Batch semua request dengan key yang sama ke satu backend call
Cache hasil per key — request berulang tidak menghitung lagi

HTTP BATCHING EQUIVALENT

Aggregate IDs, kirim satu request batch ke backend
Gunakan cache layer (Redis) untuk hasil yang relatif statis
Pattern ini sama dengan database batch query — konsep universal

Lab ini: database-level batch via GetPostsByAuthorIDs

DataLoader: application-level batch via IN(...) equivalent

Kedua pola: kumpulkan, batch, satukan di memori

Batch and cache: collect requests across a tick, fire one call, cache per key.

Bahaya Mapping-Level Eager Loading

FetchType.EAGER dan semacamnya memuat semua relasi — memory bloat

MASALAH

JPA FetchType.EAGER: memuat semua relasi otomatis di setiap query
Tidak ada kontrol selektif — over-fetching terjadi
Memory bloat: entitas tidak relevan ikut ter-load

PERBEDAAN KRITIS

Mapping-level: deklaratif di class — selalu eager
Query-level: eksplisit per query — kontrol penuh
Lab ini mendemonstrasikan query-level — solusi yang benar

Entity Author dengan @OneToMany EAGER

Setiap kali query Author, semua Post ikut ter-load

Bayangkan: 100 author, masing-masing 1000 post = 100K post loaded

Eager at the mapping level loads everything everywhere — control loading per query.

Trade-off: Cartesian Product Explosion

Eager loading via JOIN menyebabkan hasil dikali jumlah child records

PHENOMENA

Parent (1) JOIN Child (N) → hasil = 1 x N rows
Untuk 1 author dengan 50 post → 50 rows yang sama untuk author
Dengan multiple one-to-many → cartesian explosion paralel

IMPACT

Network bandwidth: data yang dikirim jauh lebih besar
Memory: result set di aplikasi jauh lebih besar
CPU: parsing dan mapping hasil yang lebih lambat

Django: prefetch_related() — gunakan query terpisah, join di Python

EF Core: split query — eksekusi query terpisah per relasi

SQLAlchemy: selectinload() — batch select, join di memori

JOIN-based eager loading causes cartesian explosion — prefer split/batch queries.

Strategi Alternatif: Column Selection

select/pluck/values/withCount — ambil skalar, bukan entitas penuh

PERBANDINGAN

Eager loading penuh: SELECT * FROM posts WHERE ... + mapping
Column selection: SELECT title FROM posts WHERE ... + raw values
Jika hanya butuh judul atau counter — column selection jauh lebih efisien

API DI BEREBERAPA ORM

Django: values('title'), withCount('posts')
Rails: pluck(:title), posts.count
Laravel: select('title')->withCount('posts')
EF Core: Select(p => p.Title), ToQueryString

Bandwidth: jauh lebih sedikit data dikirim

Memory: tidak perlu materialisasi entitas penuh

CPU: parsing lebih cepat — tidak perlu mapper ORM

For scalars and counts, column selection beats eager loading of full entities.

Workflow Profiling-First

Langkah demi langkah: ukur → identifikasi → optimasi

TIGA LANGKAH

1. Ukur query count per request (bukan hanya durasi)
2. Identifikasi pola N+1 (lazy loading di loop, per-object API call)
3. Optimasi: batch eager loading, column selection, atau caching

TOOLING

Django: django-nplusone, django-debug-toolbar
Rails: bullet gem
Laravel: beyondcode/laravel-query-detector
EF Core: Microsoft.Extensions.Logging + query tag

Query count test assertion di CI pipeline

Threshold maksimal query per endpoint

Strict loading mode untuk mencegah lazy loading tak disadari

Measure first, optimize second — query count profiling before duration.

Eager Loading di Berbagai ORM

API naming berbeda, konsep dasar sama

ORM	Method	Type	SQL
Django	select_related()	JOIN	1 query
Django	prefetch_related()	Split	2 queries
Rails	includes()	Auto-switch	1/2
Laravel	with()	Split	2 queries
EF Core	Include()	JOIN	1 query
EF Core	ThenInclude()	JOIN	1 query

select_related: JOIN — risiko cartesian explosion
prefetch_related: split query — aman dari explosion
includes: Rails memilih otomatis (smart)
Lab ini: split query via GetPostsByAuthorIDs
Mapping ini setara dengan prefetch_related / split query
Bukan JOIN-based eager loading — menghindari cartesian explosion

ORMs agree on the concept — split/batch to avoid cartesian explosion, JOIN to minimize queries.

Evaluasi: N+1 vs Eager Loading

Perbandingan kuantitatif dan kualitatif

Dimensi	N+1 (Lazy)	Eager (Batch)
Jumlah query	N+1	2
Skalabilitas	Linear (O(N))	Konstan (O(1))
Memory overhead	Minimal	Map building
Risk cartesian	Tidak ada	JOIN = ya (hindar)
Readability	Intuitive	Perlu pemahaman
Over-fetching	Hanya per item	Semua per satu

N=3: N+1=4	Eager=2
N=100: N+1=101	Eager=2
N=1000: N+1=1001	Eager=2
N=10000: N+1=10001	Eager=2

Gunakan eager loading untuk data yang pasti dibutuhkan
Gunakan column selection untuk skalar/count
Gunakan lazy loading hanya untuk data opsional/relasional

Trade intuition for correctness — eager batch wins on query count and stays O(1).

Studi Kasus: GET /api/work-orders

Dari 6 query di dev menjadi 101 di prod — kisah nyata N+1

SCENARIO AWAL

Endpoint: GET /api/work-orders
Data uji dev: 5 work order, 1 item each = 6 query
Response time: <10ms — terlihat cepat
Lolos deployment ke production

SCENARIO PRODUKSI

Production: 100 work order, 5 item each
N+1: 1 + 100 = 101 query seril
Response time: 2.4 detik (ilustratif)
Connection pool exhausted — request pending

Batched query: WHERE work_order_id IN (...) → 2 query

Column selection: hanya ambil kolom yang perlu

Pagination: tidak load 100 work order sekaligus

Query count assertion di test — mencegah regresi

Dev data hides prod pain — assert query counts for every realistic data size.

Checklist Audit Performa Data Access

9 poin verifikasi sebelum deploy ke production

CHECKLIST

[ ] Profiling query count sebelum deploy
[ ] Audit lazy loading dalam loop — ganti ke batch/eager
[ ] Eager loading per-query, bukan per-mapping (hindari EAGER global)
[ ] Cek cartesian explosion pada relasi one-to-many/many-to-many
[ ] Gunakan column selection (pluck/values) untuk skalar
[ ] Pagination untuk hasil besar
[ ] Query count assertion di unit/integration test
[ ] Pertimbangkan strict loading mode
[ ] Query count test assertion di CI pipeline

ALAT BANTU

django-nplusone — deteksi N+1 di Django
bullet gem — deteksi N+1 di Rails
laravel-query-detector — deteksi di Laravel
EF Core logging — tracking query patterns

Jika query count meningkat linear dengan N → ada N+1

Jika query count konstan → batching sudah tepat

Query count assertion = automated guardrail

Nine checkpoints cover detection, batching, and prevention — assert query counts in CI.

Batasan Implementasi Lab 22

Lab ini adalah simulasi konseptual — bukan produksi database asli

BATAS 1: IN-MEMORY MOCK STORE

Store menggunakan slice in-memory, bukan database nyata
Query counter hanya counter — tidak ada latency jaringan nyata
Angka 4 query vs 2 query adalah verifikasi logik, bukan benchmark

BATAS 2: KONEKSI KE ORM

Lab implementasi manual, bukan integrasi dengan Django/Rails/EF Core
API batch (GetPostsByAuthorIDs) merepresentasikan konsep IN(...)
ORM-level lazy vs eager loading tidak langsung ditunjukkan di sini

Angka 2.4s pada studi kasus adalah angka ilustratif

Dataset hanya 3 author — produksi bisa ribuan

Lab membuktikan konsep query count, bukan performa absolut

This lab proves query-count reduction conceptually — real-world latency varies.

Prinsip Desain Akses Data yang Benar

Tiga prinsip dasar mencegah N+1 secara sistematis

PRINSIP 1: PROFILING- FIRST

Hitung query count sebelum optimasi
Tooling: ORM query logger, APM, slow query analysis
CI/CD: query count threshold assertion

PRINSIP 2: BATCH, JANGAN LOOP

Kumpulkan ID → satu query IN(...) → join di memori
Pola ini universal: database, HTTP, GraphQL
Query-level eager loading — kontrol per query

select/pluck/values/withCount — ambil skalar

Hindari materialisasi entitas penuh jika tidak dipakai

Reduksi bandwidth, memory, dan CPU parsing

Three principles: measure first, batch not loop, select only what you need.

Ringkasan Lab 22

N+1 is a pattern, not a layer — batch all IDs into ONE query, assert count in tests.

KEY RESULTS

N+1: 1 + N query (linear dengan record count)

Eager: 2 query (konstan, terlepas dari N)

Test assertion: query count verified, deep equality confirmed

REPOSITORY

https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem

Lukman / lukman-ss

MENTAL MODEL

N+1 is a pattern, not a layer — it strikes wherever I/O runs in a loop.

Batch all related identifiers into a single IN(...) query.

Assert query counts in unit tests — guardrails don't regress silently.

RESOURCES

Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem

Commit SHA: 6649acb946725e993e78f99ec47833a5498cc870

Run demo: go run ./cmd/demo

Run tests: go test -v -race ./...